

Name: Jiya Yogesh Varma

PRN: 202501050023

Division: Chemical

Batch: CH2

Course: EDS



3.1.1. Numpy Array Operations

18.59



Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c , representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

Test case 1

3 4
1 2 3 4
5 6 7 8
9 10 11 12
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]]
2
(3, 4)
12

Test case 2

0 0
[]
2
(0, 0)
0

Explorer

numpyarr.py



Submit

Debugger

```
1 import numpy as np
2
3 # write your code here...
4 rows,cols=map(int,input().split())
5
6 elements=[]
7 for _ in range(rows):
8     elements.extend(map(int,input().split()))
9
10 array=np.array(elements).reshape(rows,cols)
11
12 print(array)
13
14 print(array.ndim)
15
16 print(array.shape)
17
18 print(array.size)
```

Average time

0.028 s

28.00 ms

Maximum time

0.067 s

67.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1 67 ms

Debug

Expected output

Actual output

3 4

3 4

1 2 3 4

1 2 3 4

5 6 7 8

5 6 7 8

< Prev

Reset

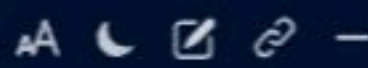
Submit

Next >



3.2.1. Numpy: Matrix Operations

16:48



The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** (`matrix_a + matrix_b`)
2. **Subtraction** (`matrix_a - matrix_b`)
3. **Element-wise Multiplication** (`matrix_a * matrix_b`)
4. **Matrix Multiplication** (`matrix_a . matrix_b`)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

Test case 1

Enter Matrix A:

1 2 3

4 5 6

7 8 9

Enter Matrix B:

1 1 1

1 1 1

1 1 1

Addition (A + B):

[[2 3 4]]

[[5 6 7]]

[[8 9 10]]

Subtraction (A - B):

[[0 1 2]]

[[3 4 5]]

[[6 7 8]]

Element-wise Multiplication (A * B):

[[1 2 3]]

[[4 5 6]]

[[7 8 9]]

A dot B:

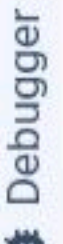
[[6 6 6]]



matrixOpe...



Submit



```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a =
6     np.array([list(map(int,
7 input().split())) for i in
8 range(3)])
9
10
11 # Addition
12 addition_result = matrix_a +
13 matrix_b
14 print("Addition (A + B):")
15 print(addition_result)
16
17 # Subtraction
18 Subtraction_result = matrix_a -
19 matrix_b
20 print("Subtraction (A - B):")
21 print(Subtraction_result)
22
23 # Multiplication (element-wise)
24 elementwise_multiplication_resu
25 lt = matrix_a * matrix_b
26 print("Element-wise
27 Multiplication (A * B):")
28 print(elementwise_multiplicatio
29 n_result)
30
31
32 # Matrix multiplication (dot
33 product)
34 Matrix_multiplication_result =
35 np.dot(matrix_a, matrix_b)
36 print("A dot B:")
37 print(Matrix_multiplication_res
38 ult)
39
40
41 # Transpose
42 Transpose_a = matrix_a.T
43 print("Transpose of A:")
44 print(Transpose_a)
```

Average time

0.060 s

59.75 ms

Maximum time

0.081 s

81.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 81 ms

Debug

Expected output

Enter Matrix A:

1 2 3

4 5 6

Actual output

Enter Matrix A:

1 2 3

4 5 6

< Prev

Reset

Submit

Next >

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

Test case 1

Enter Array1:

123

456

789

Enter Array2:

456

789

101112

Horizontal Stack:

[123456]

[456789]

[789101112]

Vertical Stack:

[123]

[456]

[789]

[456]

[789]

[101112]

Test case 2

Enter Array1:

-1-2-3

-4-5-6

-7-8-9

Enter Array2:

101112

131415

161718

Horizontal Stack:

stacking.pySubmit

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int,
6 input().split())) for i in
7 range(3)])
8
9
10 # Horizontal stacking
11 h = np.hstack((arr1, arr2))
12 print("Horizontal Stack:")
13 print(h)
14
15 # Vertical stacking
16 v = np.vstack((arr1, arr2))
17 print("Vertical Stack:")
18 print(v)
```

Average time0.070 s70.00 msMaximum time0.106 s106.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 174 msDebug

Expected outputActual output

Enter Array1:Enter Array1:

123123

456456



3.2.3. Numpy: Custom Sequence Generation 01:46

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

- Input Format:**
- The user will input three integer values: start, stop, and step, each on a new line.
- Output Format:**
- The program should print the generated sequence based on the input values.

Sample Test Cases

Test case 1

3
10
2
[3 5 7 9]

Test case 2

10
20
30
[10]

```
1 import numpy as np
2
3 # Take user input for the
  start, stop, and step of the
  sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate sequence
9 a = np.arange(start, stop,
  step)
10
11 # Print result
12 print(a)
```

Average time	Maximum time
0.034 s	0.044 s
34.25 ms	44.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

44 ms

Debug

Expected output	Actual output
3	3
10	10
2	2



You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:

- 1. Arithmetic Operations:**
 - Compute the element-wise sum, difference, and product of the two arrays.
- 2. Statistical Operations:**
 - Calculate the mean, median, and standard deviation of array A.
- 3. Bitwise Operations:**
 - Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

Test case 1

1 2 3 4	
5 6 7 8	
Element-wise Sum: 6 8 10 12	
Element-wise Difference: -4 -4 -4 -4	
Element-wise Product: 5 12 21 32	
Mean of A: 2.5	
Median of A: 2.5	
Standard Deviation of A: 1.118033988749895	
Bitwise AND: 1 2 3 0	
Bitwise OR: 5 6 7 12	
Bitwise XOR: 4 4 4 12	

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to NumPy arrays
6     A = np.array(A)
7     B = np.array(B)
8
9     # Arithmetic Operations
10    sum_result = A + B
11    diff_result = A - B
12    prod_result = A * B
13
14    # Statistical Operations
15    mean_A = np.mean(A)
16    median_A = np.median(A)
17    std_dev_A = np.std(A)
18
19    # Bitwise Operations
20    and_result = A & B
21    or_result = A | B
22    xor_result = A ^ B
23
24    # Output results with one space between each element
25    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
26    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
27    print("Element-wise Product:", ' '.join(map(str, prod_result)))
28
29    print(f"Mean of A: {mean_A}")
30    print(f"Median of A: {median_A}")
31    print(f"Standard Deviation of A: {std_dev_A}")
32
33    print("Bitwise AND:", ' '.join(map(str, and_result)))
34    print("Bitwise OR:", ' '.join(map(str, or_result)))
35    print("Bitwise XOR:", ' '.join(map(str, xor_result)))
36
37    A = list(map(int, input().split())) # Elements of array A
```

Average time	Maximum time
0.056 s	0.113 s
56.33 ms	113.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 113 ms	Debug		
Expected output	Actual output		
1 2 3 4	1 2 3 4		
5 6 7 8	5 6 7 8		
Element-wise Sum: 6 8 10 12	Element-wise Sum: 6 8 10 12		

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: <original_array>
View array: <view_array>

- After modifying the copy:

Original array after modifying copy: <original_array>
Copy array: <copy_array>

Sample Test Cases

Test case 1

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Test case 2

99 2 3 4 5

Original array after modifying view: [99 2 3 4 5]

View array: [99 2 3 4 5]

Original array after modifying copy: [99 2 3 4 5]

Copy array: [99 88 3 4 5]

copyAndvi... Submit

1 import numpy as np

2

3 inputlist =

list(map(int,input().split(")))

4

5 # Original array

6 original_array =

np.array(inputlist)

7

8 # Create a view

9 view_array =

original_array.view()

10

11 # Create a copy

12 copy_array =

original_array.copy()

13

14 # Modify the view

15 view_array[0] = 99

16 print("Original array after

modifying view:",

original_array)

17 print("View array:",

view_array)

18

19 # Modify the copy

20 copy_array[1] = 88

21 print("Original array after

modifying copy:",

original_array)

22 print("Copy array:",

copy_array)

23

Average time0.021 s20.50 msMaximum time0.022 s22.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 122 msDebug

Expected outputActual output

10 20 30 40 50 60 70 8010 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]Original array after modifying view: [99 20 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]Copy array: [10 88 30 40 50 60 70 80]

PrevResetSubmitNext



The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- 1. Searching:** Find the indices where search_value appears in array1 and print these indices.
- 2. Counting:** Count how many times count_value appears in array1 and print the count.
- 3. Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- 4. Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

Sample Test Cases

Test case 1

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

[3 3 3 4 4 4]

[1 1 1 2 2 2]

Test case 2

1 2 3 4 5

Value to search: 6

Value to count: 6

Value to add: 6

[]

```
1 import numpy as np
2
3 # Input array from the user
4 array1 =
5 np.array(list(map(int,
6 input().split()))))
7
8 # Searching
9 search_value =
10 int(input("Value to search: "))
11 count_value = int(input("Value
12 to count: "))
13 broadcast_value =
14 int(input("Value to add: "))
15
16 # Find indices where value
17 matches in array1
18 indices = np.where(array1 ==
19 search_value)
20 print(indices[0])
21
22 # Count occurrences in array1
23 count = (array1 ==
24 count_value).sum()
25 print(count)
26
27 # Broadcasting addition
28 result_array = array1 +
29 broadcast_value
30 print(result_array)
31
32 # Sort the first array
33 sorted_array = np.sort(array1)
34 print(sorted_array)
```

Average time

0.048 s

47.75 ms

Maximum time

0.087 s

87.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 38 ms

Debug

Expected output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Actual output

1 1 1 2 2 2

Value to search: 1

Value to count: 2



Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90**

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv",
4               delimiter=',', skiprows=1)
5
6 # 1
7 print("All student
8 Details:\n", a)
9
10 # 2
11 print("Total Students:",
12       a.shape[0])
13
14 # 3
15 print("All Student Roll Nos",
16       a[:, 0])
17
18 # 4
19 print("Subject 1 Marks", a[:,
20                               1])
21
22 # 5
23 print("Min marks in Subject
24 2", np.min(a[:, 2]))
25
26 # 6
27 print("Max marks in Subject
28 3", np.max(a[:, 3]))
29
30 # 7
31 print("All subject marks:",
32       a[:, 1:])
33
34 # 8
35 total = np.sum(a[:, 1:],
36               axis=1)
37 print("Total Marks", total)
38
39 # 9
40 print(np.round(np.mean(a[:,
41                           1:],
42                       axis=1), 1))
43
44 # 10
45 print("Average Marks of each
46 subject",
47       np.round(np.mean(a[:,
48                           1:],
49                       axis=0), 1))
50
51 # 11
52 print("Average Marks of S1
53 and S2",
54       np.round(np.mean(a[:,
55                           1, 2],
56                       axis=0), 1))
```

Average time

0.057 s

57.00 ms

Maximum time

0.057 s

57.00 ms

1 out of 1 shown test case(s) passed

Test case 1 57 ms

Debug

≡

📄

^

Expected output

Actual output

All student Details:

All student Details:

[[301. 67. 77. 88.]

[[301. 67. 77. 88.]

[302. 78. 88. 77.]

[302. 78. 88. 77.]

[303. 45. 56. 89.]

[303. 45. 56. 89.]